El cartero chino, explicado fácil

Cómo la app convierte el mapa en un grafo y cómo calcula la ruta más corta que pasa por todas las calles.
Matemáticas Discretas · Proyecto Cartero Chino (Barranquilla y Soledad)

> **El problema en una frase**
> Un cartero sale de la oficina, tiene que pasar **por todas las calles** de su zona al menos una vez y volver a la oficina. ¿Cuál es el recorrido **más corto** posible?

Si cada calle se pudiera recorrer exactamente una vez, la respuesta sería fácil: la suma de todas las calles. El problema es que casi nunca se puede: en algún momento el cartero queda "atrapado" y le toca **repetir** calles para seguir. Lo que hace la app es encontrar **qué calles repetir para que lo repetido sea lo mínimo**.

Todo el trabajo se divide en dos partes:

Parte	Qué hace	En palabras simples
A. Del mapa al grafo	Descarga las calles reales y las convierte en puntos y líneas	"Pasar el mapa a un dibujo del tablero"
B. La ruta del cartero	Busca los cruces problemáticos, los une por el camino más corto y arma el recorrido	"Decidir qué calles repetir y en qué orden caminar"

Lo único que necesitas saber de grafos

Un **grafo** es solo un dibujo con **puntos** y **líneas** que unen puntos:

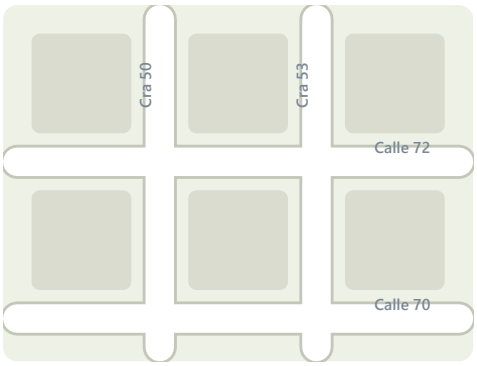
Palabra del curso	Qué es en la app	Ejemplo
Vértice (punto)	Un cruce de calles (una esquina)	Calle 72 con Carrera 53
Arista (línea)	Un tramo de calle entre dos cruces seguidos	La Calle 72 entre la Cra 50 y la Cra 53
Peso de la arista	Cuántos metros mide ese tramo	110 m
Grado de un vértice	Cuántos tramos salen de esa esquina	Un cruce normal en "+" tiene grado 4

La fórmula de la longitud de la ruta es solo **sumar los pesos** de las aristas que se recorren:

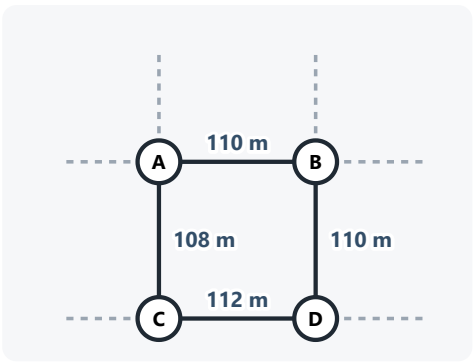
$$L = w(e_1) + w(e_2) + \cdots + w(e_m) = \sum_{i=1}^{m} w(e_i)$$

Se lee: "la longitud L es la suma de los pesos (metros) de todas las aristas recorridas". El símbolo Σ solo significa "sumar".

Parte A · Cómo el mapa se convierte en grafo

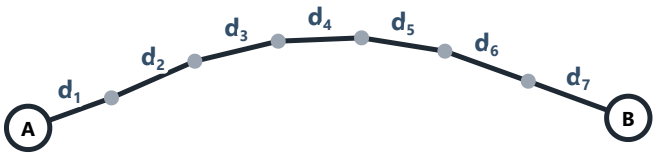


1. Las calles reales (OpenStreetMap)



2. El grafo: cruces = vértices, tramos = aristas

1. **Descargar las calles.** Cuando seleccionas un área (rectángulo o polígono), la app trae las calles de **OpenStreetMap**, un mapa libre parecido a Google Maps. Allá cada calle viene guardada como una **lista de puntos GPS** (latitud, longitud) uno detrás de otro, más datos como el nombre y si es de un solo sentido. Para Barranquilla y Soledad la app ya trae las calles guardadas, así no depende de internet lento.
2. **Encontrar los cruces (vértices).** Si un mismo punto GPS aparece en **dos o más calles**, ahí las calles se cruzan: ese punto es un **vértice**. Los finales de calle (calles ciegas) también son vértices. Los demás puntos de la lista solo sirven para dibujar la forma de la calle (sus curvas).
3. **Cortar la calle en tramos (aristas).** Cada calle se corta en cada cruce. El pedazo entre dos cruces seguidos es una **arista**. Una calle larga como la Murillo queda partida en muchas aristas, una por cuadra.
4. **Medir cada tramo (peso).** El tramo puede ser curvo, así que la app mide la distancia entre cada par de puntitos seguidos y los **suma**:



peso del tramo A–B = d1 + d2 + … + d7 (en metros)

5. **Limpiar.** Se quedan solo los tramos con sus dos extremos **dentro del área** que marcaste (las líneas grises punteadas del dibujo quedan por fuera). Si un punto no es un cruce de verdad (por ejemplo, donde una calle solo cambia de nombre), se borra y sus dos tramos se unen en uno. Las calles de **un solo sentido** se guardan con su flecha. Al final se numeran los vértices de norte a sur: **v1** es la esquina de más arriba a la izquierda.

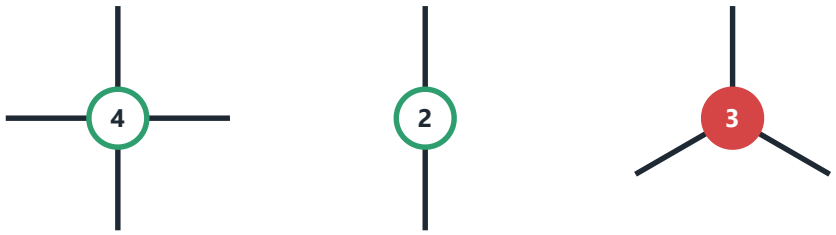
Si el profe pregunta: ¿cómo mide la distancia entre dos puntos GPS?
Usa la **fórmula de Haversine**, que mide distancias sobre una esfera (la Tierra es redonda), con radio $R = 6\,371$ km:

$$d = 2R \arcsin\sqrt{\sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)}$$

φ es la latitud y λ la longitud de cada punto; Δ significa "diferencia". No hace falta memorizarla: para distancias de una cuadra da prácticamente lo mismo que medir con regla sobre el mapa.

Parte B · La idea clave: entrar y salir

Imagina que caminas sin repetir ninguna calle. Cada vez que **llegas** a una esquina por una calle, tienes que **salir** por otra calle distinta. O sea, en cada esquina las calles se van gastando **de a dos**: una para entrar, otra para salir.



4 calles: entra-sale, entra-sale ✓ 2 calles: entra-sale ✓ 3 calles: sobra una ✗

Por eso, si quieres pasar por todas las calles **sin repetir** y volver al inicio, **todas las esquinas deben tener un número par de calles**. Si una esquina tiene 3 calles: entras por una, sales por otra… y la tercera te obliga a volver a pasar por ahí usando una calle ya usada.

> **Teorema de Euler (lo único que hay que recordar)**
> Un grafo conectado se puede recorrer pasando **una sola vez por cada arista** y volviendo al inicio **si y solo si todos sus vértices tienen grado par**. A ese recorrido se le llama **circuito de Euler**.

Los vértices impares siempre vienen en parejas

Cada calle tiene **dos puntas**. Si cuentas las calles en cada esquina y sumas todo, cada calle se cuenta dos veces (una por cada punta):

$$\sum_{v} \text{grado}(v) = 2 \cdot (\text{número de aristas})$$

Se lee: "la suma de los grados de todos los vértices es el doble del número de aristas" (teorema del apretón de manos).

El doble de algo siempre es **par**. Para que una suma dé par, los números impares tienen que venir en cantidad par. Conclusión: **los vértices de grado impar siempre son 2, 4, 6…** nunca un número impar de ellos. Por eso se pueden **emparejar**.

> **Entonces, ¿qué hace el cartero chino?**
> Si hay esquinas impares, el cartero va a tener que repetir calles. Repetir una calle es como **dibujar una copia** de esa arista, y eso le suma 1 al grado de sus dos puntas. Así que la estrategia es: **repetir calles que unan los impares de a parejas**, para que todos queden pares, y escoger las parejas y los caminos que **sumen menos metros**.

El método paso a paso con el ejemplo del tablero

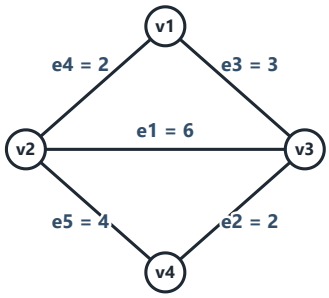
Es el mismo grafo que vimos en clase (29 de octubre). La app lo tiene en "Ejemplos de clase".

● vértice impar ● camino más corto ● calle repetida (copia)

1 **Sumar todas las calles.** Esto se camina sí o sí, porque hay que pasar por todas:

$$6 + 2 + 3 + 2 + 4 = 17$$

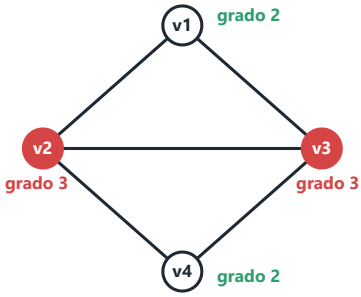
Si todo fuera par, la respuesta sería 17 y listo.



2 **Buscar los vértices impares.** Se cuenta cuántas líneas toca cada punto:

Vértice	v1	v2	v3	v4
Grado	2	**3**	**3**	2

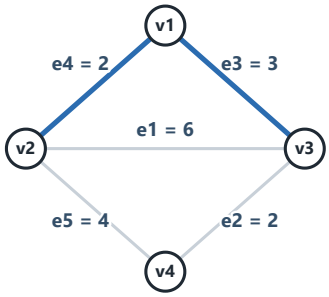
Comprobación: $2 + 3 + 3 + 2 = 10 = 2 \cdot 5$ aristas ✓. Los impares son **v2 y v3**: no hay circuito de Euler todavía.



3 **Unir los impares por el camino más corto.** ¿Cuál es la forma más barata de ir de v2 a v3?

Opción	**Metros**
Directo por e1	6
Por v1 (e4 + e3)	**2 + 3 = 5**
Por v4 (e5 + e2)	4 + 2 = 6

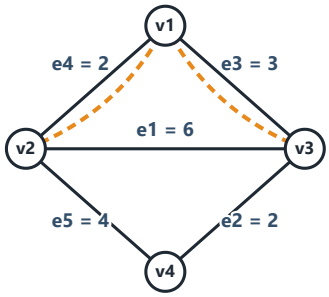
Gana **pasar por v1**: cuesta 5. Ojo: no siempre gana la línea directa.



4 **Repetir ese camino.** Se copian e4 y e3. Ahora v2 y v3 tienen grado 4 y v1 también (tenía 2, le llegan 2 copias). **Todos pares ✓.**

$$L = 17 + 5 = 22$$

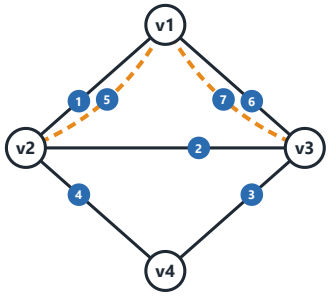
calles obligatorias + calles repetidas = recorrido mínimo



5 **Caminar el circuito de Euler.** Como todo es par, ya se puede recorrer cada línea (incluidas las copias) una vez y volver:

v1 → v2 → v3 → v4 → v2 → v1 → v3 → v1

Los números azules muestran el orden. Total: $2 + 6 + 2 + 4 + 2 + 3 + 3 = 22$ ✓



Qué hace la app por dentro (los 3 algoritmos)

Con 4 vértices se hace a ojo. En un barrio real hay cientos o miles de esquinas, así que la app usa tres algoritmos, uno para cada paso difícil:

1. Dijkstra: el camino más corto entre dos esquinas (paso 3)

Funciona como una mancha de agua que se riega desde el punto de salida y siempre avanza primero por donde está **más cerca**:

1. La salida tiene distancia 0; todas las demás, "infinito" (aún no se sabe).
2. Se toma la esquina **más cercana** que no se haya cerrado y se miran sus vecinas: si llegar por aquí es más corto, se anota la nueva distancia.
3. Se cierra esa esquina (su distancia ya es definitiva) y se repite hasta llegar al destino.

Ronda	Se cierra	v1	v3	v4	Qué pasó
0	v2 (0)	2	6	4	Vecinas de v2
1	v1 (2)	2	**5**	4	Por v1 llego a v3 con 2+3 = 5 < 6: se mejora
2	v4 (4)	2	5	4	Por v4 llegaría con 4+2 = 6: no mejora
3	**v3 (5)**	**distancia final v2→v3 = 5**			**Camino: v2 → v1 → v3**

2. Emparejamiento mínimo: quién va con quién (cuando hay más de 2 impares)

Si hay 4 esquinas impares A, B, C, D, hay varias formas de armar las parejas. La app calcula con Dijkstra la distancia entre cada par y escoge las parejas que **sumen menos**:

Parejas	Distancias	Total a repetir
A–B y C–D	**3 + 4**	**7 ← la mejor**
A–C y B–D	5 + 6	11
A–D y B–C	2 + 7	9

Fíjate que la pareja más corta sola (A–D = 2) **no** está en la mejor solución: por eso no sirve ir escogiendo "la más cercana" una por una; hay que mirar la suma total. Con pocos impares (hasta 20) la app prueba todas las combinaciones de forma inteligente; con muchos (Barranquilla tiene unas 6 800 esquinas impares) usa el **algoritmo Blossom de Edmonds**, que encuentra el emparejamiento mínimo sin probar todas.

3. Hierholzer: armar el recorrido (paso 5)

Cuando ya todo es par, se arma el circuito así:

1. Sal del inicio y camina por calles no usadas hasta que vuelvas al inicio (siempre vuelves, porque todo es par).
2. Si quedaron calles sin usar, busca una esquina de tu vuelta que todavía tenga calles libres, haz otra vuelta desde ahí y **métela** en la primera.
3. Repite hasta usar todas las calles.

Detalle de la app: en cada esquina prefiere seguir **derecho** antes que girar, para que la ruta se parezca más a cómo maneja una persona.

¿Por qué la ruta de la app es la más corta posible?

Este es el argumento para defenderlo en la exposición, en tres ideas:

1. **Todas las calles se caminan al menos una vez.** Eso es fijo para cualquier ruta: en el ejemplo, 17.
2. **Cualquier ruta válida tiene que repetir calles que "arreglen" a los impares.** Cada esquina impar necesita al menos una calle repetida que llegue a ella; las calles repetidas forman caminos que unen impares de a dos.
3. **Cada uno de esos caminos mide como mínimo el camino más corto** (Dijkstra), y la app escoge **las parejas que suman menos** (emparejamiento mínimo). Entonces ninguna otra ruta puede repetir menos metros.

$$L_{\min} = \underbrace{\sum w(\text{todas las calles})}_{\text{obligatorio}} + \underbrace{\text{emparejamiento mínimo de los impares}}_{\text{lo que se repite}}$$

Tablero: 17 + 5 = 22. En la app esto aparece como "longitud de las calles" + "repetido".

Ojo con estas diferencias

- **Ruta A → B (modo tipo Waze)** es otro problema: no hay que pasar por todas las calles, solo llegar. Ahí basta con **Dijkstra**. Con varias paradas, la app además prueba en qué **orden** visitarlas para que el total sea menor.
- **Calles de un solo sentido:** en el modo A → B en carro la app las respeta (y también los giros prohibidos). En el cartero chino las trata como de doble sentido, como se ve en clase (el cartero va a pie).
- **"Moverse por toda la ciudad":** si marcas un área, la app puede usar calles de afuera solo para **pasar** (líneas verde azulado punteadas), cuando eso es más corto que repetir calles de adentro. Esa variante se llama **cartero rural**.

Resumen para repasar

Pregunta	Respuesta corta
¿Qué es un vértice en la app?	Un cruce de calles (o un final de calle).
¿Qué es una arista?	El tramo de calle entre dos cruces seguidos; su peso son los metros.
¿Cuándo no hay que repetir nada?	Cuando todas las esquinas tienen grado par (teorema de Euler).
¿Por qué los impares van en parejas?	La suma de grados es 2 × aristas, siempre par.
¿Qué calles se repiten?	Los caminos más cortos que unen a los impares por parejas, escogiendo las parejas que suman menos.
¿Qué algoritmos usa?	Dijkstra (camino más corto), Blossom (parejas), Hierholzer (recorrido).
¿Cuánto da el ejemplo de clase?	17 de calles + 5 repetidos (e4 y e3) = 22.